

Chapter 42: Greedy Algorithms

Personal Notes

You've spent chapters on Dynamic Programming — carefully building tables to explore every subproblem. Greedy algorithms are the opposite: at every step, just make the **LOCALLY OPTIMAL CHOICE** and never look back. No table. No memoization. Just a rule, applied over and over.

When greedy works, it's the fastest and cleanest possible solution. When it doesn't, it fails spectacularly — giving wrong answers that **LOOK** right. The skill is recognizing which is which. This chapter covers the classic greedy problems, how to spot them in interviews, and the trap of mistaking a DP problem for a greedy one.

1. What is a Greedy Algorithm?

A greedy algorithm builds a solution **PIECE BY PIECE**, always choosing the piece that looks best **RIGHT NOW** — with no regard for future consequences. If that local optimum leads to a global optimum, greedy wins. If not, greedy silently gives wrong answers.

The Greedy Recipe

1. Define the **ITEMS** or **STEPS** you're choosing among
2. Define a **GREEDY RULE** — which item is "best" at each step
3. Sort or organize the input so the best item is easy to pick
4. Repeatedly pick, commit, and move on — no backtracking allowed

Example: Coin change with coins {1, 5, 10, 25} for amount 63

Greedy rule: always take the **LARGEST** coin $\leq$ remaining amount.

63 → take 25 (remaining 38)
38 → take 25 (remaining 13)
13 → take 10 (remaining 3)
3 → take 1, 1, 1 (remaining 0)

Total: 6 coins. Correct? YES for US coins.
(For arbitrary coin sets — see the warning coming up.)

The core intuition: Greedy = "take the best-looking option now." It works when local optima **BUILD UP** to a global optimum without conflict. That's not always true — but when it is, greedy gives $O(n \log n)$ or better, versus DP's $O(n \times \text{states})$.

2. When Does Greedy Work?

A greedy algorithm gives the correct answer only when the problem has both:

- Greedy-choice property — a globally optimal solution can be built by choosing the local optimum at each step
- Optimal substructure — the optimal answer to the whole problem contains optimal answers to its subproblems

If BOTH hold, greedy works. If only optimal substructure holds (no greedy-choice property), you need DP instead.

The classic trap: Coin change with coins $\{1, 5, 10, 25\}$ → greedy works (US currency was designed that way). But with coins $\{1, 3, 4\}$ and amount 6, greedy takes $4+1+1 = 3$ coins, when the correct answer is $3+3 = 2$ coins. Same problem, different coin set — greedy fails silently. This is why you must PROVE greedy works before using it.

3. How to Prove Greedy Works

Two standard proof techniques — used more for confidence than for interviews:

3.1 Exchange Argument

Assume some optimal solution differs from the greedy solution. Show that you can EXCHANGE the differing choice with the greedy choice without making the solution worse. Since we can always exchange, the greedy solution is at least as good as any optimal solution.

3.2 Greedy Stays Ahead

Show that at every step, the greedy solution is AT LEAST AS GOOD as any other partial solution. Since greedy stays ahead throughout, it wins in the end too.

For interviews: You usually don't need formal proofs — but you should be able to explain WHY your greedy rule works. "I sort by end time because finishing a meeting earlier leaves more room for later ones" is a valid interview-level justification. If you can't articulate that intuition, it's a warning sign that DP might be needed.

4. Classic Problem 1 — Activity Selection

Given N activities with start and end times, select the MAXIMUM number of non-overlapping activities you can attend. This is the TEXTBOOK greedy problem.

The Greedy Rule

Sort activities by END TIME. Then pick every activity whose start $\geq$ the end of the last picked one.

Why This Works

Picking the activity that ends earliest leaves the MOST room for future activities. Any other choice ends later or equal, so it can't leave more room. Formally provable by exchange argument.

```
public int maxActivities(int[][] activities) {
    // activities[i] = [start, end]
    Arrays.sort(activities, (a, b) -> a[1] - b[1]);    // sort by end time

    int count = 0, lastEnd = Integer.MIN_VALUE;
    for (int[] a : activities) {
        if (a[0] >= lastEnd) {
            count++;
            lastEnd = a[1];
        }
    }
    return count;
}

// Time: O(n log n) for sort + O(n) for scan.
// Space: O(1) extra.
```

Dry Run

Activities: [(1,4), (3,5), (0,6), (5,7), (3,9), (5,9), (6,10)]

Sort by end time:

[(1,4), (3,5), (0,6), (5,7), (3,9), (5,9), (6,10)]

Pick (1,4) → lastEnd = 4	count = 1
Skip (3,5): 3 < 4	(starts before last end)
Skip (0,6): 0 < 4	
Pick (5,7) → lastEnd = 7	count = 2
Skip (3,9): 3 < 7	
Skip (5,9): 5 < 7	
Skip (6,10): 6 < 7	

Answer: 2 (with different picks – max is often 4:
(1,4), (5,7), later-non-overlapping, etc.)

For arr sorted differently, you might get 4 valid: (1,4)(5,7) etc.

5. Classic Problem 2 — Non-Overlapping Intervals

Given a list of intervals, find the MINIMUM NUMBER to remove so the rest don't overlap. Equivalent to Activity Selection — maximize the number kept, remove the rest.

```
public int eraseOverlapIntervals(int[][] intervals) {
    Arrays.sort(intervals, (a, b) -> a[1] - b[1]);
```

```

    int kept = 0, lastEnd = Integer.MIN_VALUE;
    for (int[] iv : intervals) {
        if (iv[0] >= lastEnd) {
            kept++;
            lastEnd = iv[1];
        }
    }
    return intervals.length - kept;
}

```

Same greedy rule, different framing. Sort by end time → keep as many as possible → return the discarded count.

6. Classic Problem 3 — Meeting Rooms (Two Variants)

6.1 Meeting Rooms I — Can Attend All?

Given intervals, check if a person can attend ALL of them without conflict.

```

public boolean canAttendMeetings(int[][] intervals) {
    Arrays.sort(intervals, (a, b) -> a[0] - b[0]); // sort by start
    for (int i = 1; i < intervals.length; i++) {
        if (intervals[i][0] < intervals[i - 1][1]) return false;
    }
    return true;
}

```

6.2 Meeting Rooms II — Minimum Rooms Needed

Given intervals, find the MIN number of rooms needed to hold every meeting. Greedy + min-heap of end times.

```

public int minMeetingRooms(int[][] intervals) {
    Arrays.sort(intervals, (a, b) -> a[0] - b[0]);
    PriorityQueue<Integer> endTimes = new PriorityQueue<>();

    for (int[] iv : intervals) {
        if (!endTimes.isEmpty() && endTimes.peek() <= iv[0]) {
            endTimes.poll(); // reuse a freed room
        }
        endTimes.offer(iv[1]);
    }
    return endTimes.size();
}

// Time: O(n log n)   Space: O(n)

```

How it works: Sort meetings by start time. For each meeting, check if ANY room has already freed up (min-heap top $\leq$ current start). If yes, reuse it (poll then push). If no, allocate a new room (just push). The heap size AT THE END = max concurrent meetings = rooms needed.

7. Classic Problem 4 — Jump Game

Given an array where each element is the max jump length from that position, determine if you can reach the last index starting from index 0.

The Greedy Rule

Track the FARTHEST index you can reach. If at any position you're beyond that reach, return false. If your reach ever $\geq$ last index, return true.

```
public boolean canJump(int[] nums) {
    int farthest = 0;
    for (int i = 0; i < nums.length; i++) {
        if (i > farthest) return false; // stuck
        farthest = Math.max(farthest, i + nums[i]);
    }
    return true;
}

// Time: O(n)   Space: O(1)
```

Jump Game II — Minimum Jumps

```
public int jump(int[] nums) {
    int jumps = 0, currentEnd = 0, farthest = 0;
    for (int i = 0; i < nums.length - 1; i++) {
        farthest = Math.max(farthest, i + nums[i]);
        if (i == currentEnd) { // must jump
            jumps++;
            currentEnd = farthest;
        }
    }
    return jumps;
}
```

Think of it like BFS in disguise — every "jump" is a level. Within each level (i from currentEnd to farthest), we find the reach of the next level.

8. Classic Problem 5 — Gas Station

Given circular gas stations with gas[i] fuel and cost[i] to reach the next station, find a starting station from which you can complete the circuit — or return -1 if impossible.

The Insight

- If total gas < total cost → impossible, return -1
- Otherwise, a valid start EXISTS. Find it in one pass.
- Whenever the running tank goes negative, RESET start to i+1 and start fresh

```
public int canCompleteCircuit(int[] gas, int[] cost) {
    int totalTank = 0, currTank = 0, start = 0;

    for (int i = 0; i < gas.length; i++) {
        int diff = gas[i] - cost[i];
        totalTank += diff;
        currTank += diff;

        if (currTank < 0) {
            start = i + 1;    // any earlier start also fails
            currTank = 0;
        }
    }
    return totalTank >= 0 ? start : -1;
}

// Time: O(n)    Space: O(1)
```

The trick is subtle: If we can't reach station j starting from station i, we ALSO can't reach j from any station between i and j (proof: those intermediate starts had even less fuel available at j). So we skip them all — try i+1... wait, no — jump directly to j+1. This is what makes the single-pass work.

9. Classic Problem 6 — Task Scheduler

Given a list of tasks and cooldown n (same task must be n intervals apart), find the minimum time to finish all tasks.

The Greedy Rule

Always schedule the MOST FREQUENT remaining task first — it dictates the total length. Use a max-heap of frequencies to always pick the most frequent, plus a cooldown queue.

```
public int leastInterval(char[] tasks, int n) {
    int[] freq = new int[26];
    for (char t : tasks) freq[t - 'A']++;

    int maxFreq = 0, countMax = 0;
    for (int f : freq) {
        if (f > maxFreq) { maxFreq = f; countMax = 1; }
        else if (f == maxFreq) countMax++;
    }

    // Formula: (maxFreq - 1) * (n + 1) + countMax
```

```
// But no less than tasks.length (if lots of variety fills idle slots)
return Math.max(tasks.length, (maxFreq - 1) * (n + 1) + countMax);
}
```

This is the elegant closed-form version. The intuition: place the most frequent task first at intervals of $(n+1)$. Add tail slots for other tasks with the same max frequency.

10. Classic Problem 7 — Huffman Coding (Preview)

Given character frequencies, build a prefix-free code that minimizes total encoded length. Huffman's greedy algorithm: repeatedly combine the two LOWEST-frequency nodes into a subtree.

```
public int huffmanCost(int[] frequencies) {
    PriorityQueue<Integer> pq = new PriorityQueue<>();
    for (int f : frequencies) pq.offer(f);

    int cost = 0;
    while (pq.size() > 1) {
        int a = pq.poll();
        int b = pq.poll();
        int merged = a + b;
        cost += merged;
        pq.offer(merged);
    }
    return cost;
}

// Time:  $O(n \log n)$ 
```

Used in file compression (ZIP), image formats (JPEG), and communication protocols. The greedy choice: always merge the two least-frequent — pushing them deeper in the tree costs less.

11. Classic Problem 8 — Fractional Knapsack

Given items with values and weights, and a capacity W , maximize total value in the knapsack. Unlike 0/1 Knapsack (from Ch 32), here items can be TAKEN FRACTIONALLY.

The Greedy Rule

Sort items by VALUE-PER-UNIT-WEIGHT (density). Take as much as possible of the highest-density item, then move to the next.

```
class Item {
    int weight, value;
}

public double fractionalKnapsack(Item[] items, int capacity) {
    Arrays.sort(items, (a, b) -> Double.compare(
        (double) b.value / b.weight, (double) a.value / a.weight
    ));
```

```

double totalValue = 0;
for (Item it : items) {
    if (capacity >= it.weight) {
        totalValue += it.value;
        capacity -= it.weight;
    } else {
        totalValue += (double) it.value * capacity / it.weight;
        break;
    }
}
return totalValue;
}

```

Only works for FRACTIONAL — not 0/1: 0/1 Knapsack (each item taken 0 or 1 times) is NOT solvable by greedy. You must use DP. The greedy fails because you can't always afford the highest-density item — you might have to skip it and take a lower-density one instead. Only fractional knapsack has the greedy-choice property.

12. Classic Problem 9 — Assign Cookies

Given children with greed factors and cookies with sizes, maximize the number of content children. A child is content if they get a cookie $\geq$ their greed factor.

```

public int findContentChildren(int[] g, int[] s) {
    Arrays.sort(g);
    Arrays.sort(s);

    int child = 0, cookie = 0;
    while (child < g.length && cookie < s.length) {
        if (s[cookie] >= g[child]) child++;
        cookie++;
    }
    return child;
}

```

Sort both arrays, then use two pointers. Give the smallest cookie to the least-greedy child that can be satisfied. Any other pairing wastes a bigger cookie or leaves a child unsatisfied.

13. Common Greedy Problem Patterns

Problem shape	Greedy Rule
Max non-overlapping intervals	Sort by END time, greedily pick
Min removals for non-overlap	Same as above, count discards
Meeting rooms needed	Sort by start + min-heap of end times

Problem shape	Greedy Rule
Can attend all meetings?	Sort by start, check consecutive overlap
Reach the end (Jump Game)	Track farthest reach in one pass
Min jumps to reach end	BFS-style with reach boundaries
Gas station circuit	Track running tank + reset trick
Task scheduling with cooldown	Most-frequent-first greedy + formula
Minimize encoding cost	Huffman: merge lowest-freq pair
Maximize items in bag (fractional)	Sort by value/weight ratio
Assign cookies / bikes / matches	Sort both sides + two-pointer greedy
Minimize platform / room count	Sort events, count concurrent
Reorganize / rearrange characters	Most frequent first (heap-based)

14. Greedy vs Dynamic Programming

Aspect	Greedy	Dynamic Programming
Approach	Local optimum at each step	Explore all subproblems
Space	Usually $O(1)$ or $O(n)$ for sorting	$O(\text{states})$ — often $O(n^2)$
Time	Often $O(n \log n)$ — dominated by sort	$O(\text{states} \times \text{transitions})$
Correctness	Only if greedy-choice property holds	Always correct (given right recurrence)
Signal	Sort + one-pass — feels natural	Overlapping subproblems, need memo
Failure mode	Silently wrong answer	Slow but correct

The decision rule: If greedy works, it's dramatically faster and simpler than DP. But you **MUST** verify it works on your problem. Test with small counterexamples — if a locally-optimal choice ever produces a globally-worse answer, greedy is wrong and you need DP. When in doubt, DP is the safer bet.

15. Common Mistakes

Mistake 1: Assuming greedy works without checking

Classic trap: coin change with arbitrary denominations. Or 0/1 knapsack. Both LOOK greedy-friendly but silently fail. Always test with 2-3 tricky inputs before committing.

Mistake 2: Wrong sorting criterion

```
// Activity Selection – MUST sort by END time
Arrays.sort(activities, (a, b) -> a[0] - b[0]); // x sorts by START
Arrays.sort(activities, (a, b) -> a[1] - b[1]); // ✓ sort by END
```

Mistake 3: Subtraction comparators overflowing

```
Arrays.sort(intervals, (a, b) -> a[1] - b[1]); // x can overflow for
large values
Arrays.sort(intervals, Comparator.comparingInt(a -> a[1])); // ✓ safe
```

Mistake 4: Forgetting to reset in greedy scans

In Gas Station, when currentTank goes negative, RESET both start and currentTank. Forgetting to reset currentTank means you carry over the negative debt — silently wrong result.

Mistake 5: Confusing greedy with brute force

Greedy is not "try things and see." It's a deterministic rule that must be executed exactly as specified. Any "try all possibilities" logic is backtracking or DP, not greedy.

Mistake 6: Sorting when you shouldn't

Some greedy problems (like Gas Station and Jump Game) work on the ORIGINAL order. Sorting destroys the sequential dependency and gives wrong answers. Read the problem carefully to know if order matters.

Mistake 7: Missing edge cases

Empty array. Single element. All identical elements. Extreme values. Greedy solutions often break silently on edge cases because there's no explicit "invalid state" — always test these.

16. Best Practices

- Try greedy FIRST for optimization problems — it's cheaper if it works
- Verify with 3-5 hand-picked test cases before committing
- For interval problems, always ask: sort by START or END?
- Use Comparator.comparingInt to avoid subtraction overflow
- When greedy is wrong on your first try, don't force it — pivot to DP
- Combine with a min-heap when you need "minimum resource so far" info

- For counting problems, an int running counter is often enough — no complex state
- Explain your greedy CHOICE aloud in interviews — proves you understand why it works

17. Quick Reference

Concept	Meaning
Greedy Algorithm	Take the best-looking option now, never revisit
Greedy-choice property	Local optimum → global optimum
Optimal substructure	Whole answer = combination of sub-answers
Exchange argument	Prove greedy by swapping choices without harm
Common sort	By end time (intervals), by ratio (knapsack)
Heap companion	Min-heap for tracking minimum resource
Two-pointer companion	Sort both sides, pair greedily
Formula shortcut	Some greedy problems have $O(n)$ closed forms
Failure mode	Silently gives wrong answers — must verify
When to prefer	Faster than DP if it works; try first

18. Practice Problems

Practice in order — foundational first, then interval-heavy, then trickier.

Foundational

5. Assign Cookies (LeetCode 455).
6. Best Time to Buy and Sell Stock II — unlimited transactions (LeetCode 122).
7. Lemonade Change (LeetCode 860).
8. Maximum Units on a Truck (LeetCode 1710).

Interval Family

9. Non-Overlapping Intervals (LeetCode 435).
10. Meeting Rooms (LeetCode 252).
11. Meeting Rooms II (LeetCode 253).
12. Minimum Number of Arrows to Burst Balloons (LeetCode 452).
13. Merge Intervals (LeetCode 56).

- 14. Insert Interval (LeetCode 57).
- 15. Car Pooling (LeetCode 1094).

Jump / Reach

- 16. Jump Game (LeetCode 55).
- 17. Jump Game II — minimum jumps (LeetCode 45).
- 18. Gas Station (LeetCode 134).
- 19. Video Stitching (LeetCode 1024).

Frequency & Scheduling

- 20. Task Scheduler (LeetCode 621).
- 21. Reorganize String (LeetCode 767).
- 22. Longest Happy String (LeetCode 1405).

Advanced

- 23. Candy — distribute candies with rating rules (LeetCode 135).
- 24. Partition Labels (LeetCode 763).
- 25. Boats to Save People (LeetCode 881).
- 26. Two City Scheduling (LeetCode 1029).
- 27. Minimum Number of Refueling Stops (LeetCode 871).
- 28. Wiggle Subsequence (LeetCode 376).

19. Final Takeaway

Greedy is the fastest strategy in DSA — when it works. The catch is that "when it works" is a subtle property, and greedy fails SILENTLY when it doesn't. That's why greedy is more of an art than a formula: you develop intuition by seeing 15-20 problems and internalizing which local rules lead to global optima.

For interviews, master five problems: Activity Selection (sort by end time), Meeting Rooms II (heap + sort), Jump Game (track farthest reach), Gas Station (reset trick), and Task Scheduler (max-frequency-first). Those five hit every major greedy pattern. When you meet a new problem, try greedy first; if a small counterexample breaks it, fall back to DP.

Key Takeaway: Greedy = local optimum at every step, no backtracking. Requires both greedy-choice property AND optimal substructure. Common companions: sorting (by end time for intervals, by ratio for knapsack), min-heap (for tracking minimums), two pointers (for pairing). Try greedy FIRST — it's faster and simpler if it works. Verify with small counterexamples. When greedy silently fails, DP is your safety net.

20. What's Next?

With greedy covered, the four big algorithm families (Divide-and-Conquer, DP, Backtracking, Greedy) are complete. Remaining specialized topics:

- Binary Search Patterns in Depth — search on the answer space, rotated arrays
- Segment Tree / Fenwick Tree — range queries WITH point updates
- Advanced Strings — KMP, Rabin-Karp, Z-algorithm, suffix arrays
- Advanced Graphs — Bellman-Ford, Floyd-Warshall, articulation points, bridges
- Number Theory basics — modular arithmetic, GCD tricks, primes
- Randomized Algorithms — quickselect, reservoir sampling
- System Design fundamentals — the natural next step